

JAVA

2025 march, currently Java 24v
- James Gosling (Sun Microsystems) 2014 → Java 8
(but now owned by Oracle)

features

1) platform independent

→ Java compiler is responsible for converting source code to byte code (.class file). This .class file can be run on any platform such as windows, mac etc. That's why it is $P \perp$ (WORA)

2) simple - becoz the complex features such as pointers, operator loading are removed in Java.
→ also becoz, the unreferenced objs or unused obj are automatically removed by GC.

3) Secure - memory management, absence of pointers which prevents memory related issues.

4) Architectural-neutral - if the architecture of the SW have any modifications or upgradations, the byte code still runs on that machine.

5) Object oriented - centered around the concept of objects.

6) Interpreted & High performance through JIT.

7) Robust - strong becoz of exception handling, GC.

8) Multithreaded - allows 'n' no. of tasks at same time by sharing one memory.

9) Distributed - access resources over the network by using Remote method Invocation (RMI).

10) Dynamic - loads classes & libraries only when needed not all at once. This makes program faster & more efficient.

Types of servers - Apache, IIS (Internet Info server),
lighttpd, Tigsaw server. web servers

→ Application servers - Java EE / Jakarta EE
→ Database servers.

Java 8 features

1. Lambda expressions: concise functional code using $\rightarrow$.
 2. Functional interfaces: single-method interface ^(with one abstract method).
 3. Interface can have default methods & static methods.
 4. Optional class: to avoid null and handle optional (nullable) values.
 5. New Date & Time API in java.time package.
 6. Stream API - for processing collections of data in declarative & pipeline way (filter, map, reduce).
- ✚ Method references - shortcut for calling a method using "::".

⇒ var-arg() - A variable-length argument is specified by 3 periods (...).

→ declare a method which can take variable no. of arguments.

eg:

```
fun (int... a) {  
    }
```

eg:

```
fun (10); ✓
```

```
fun (); ✓
```

```
fun (10, 20); ✓
```

```
fun (10, 20, 30, 40, 50); ✓
```


JVM

1) Virtual machine that runs java bytecode.

2) class loader, bytecode verifier, runtime engine, JIT compiler.

3) platform dependent but bytecode is platform independent.

4) executes .class file

JRE

→ package to run java programs

→ JVM + libraries + runtime files

→ platform dependent (OS specific)

→ provides environment to run java

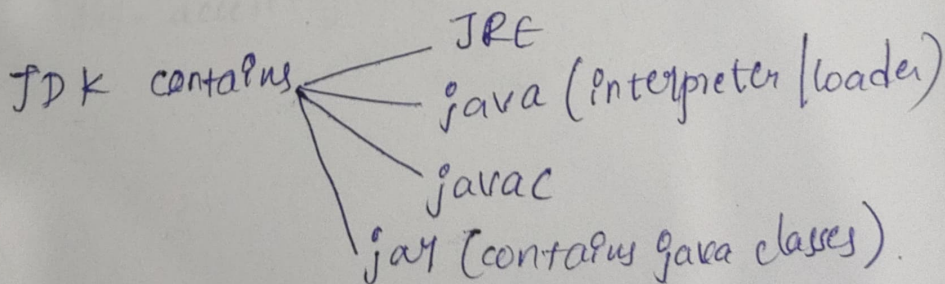
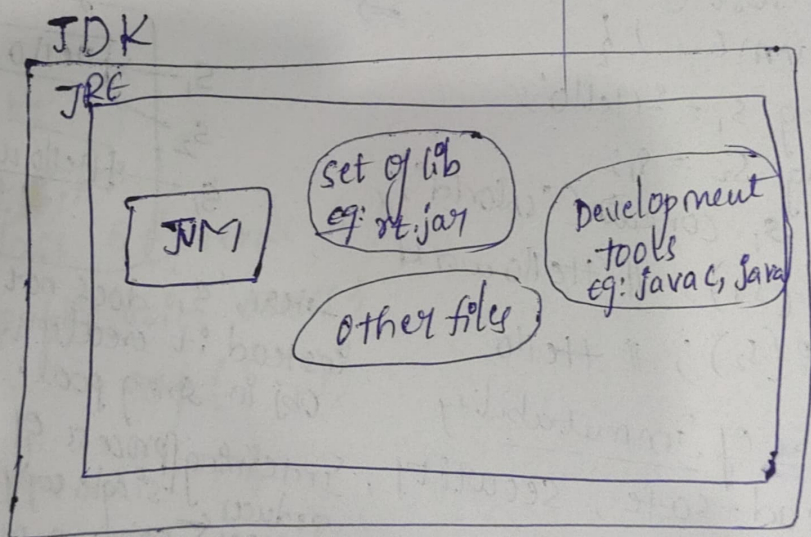
JDK

→ used to develop java applications.

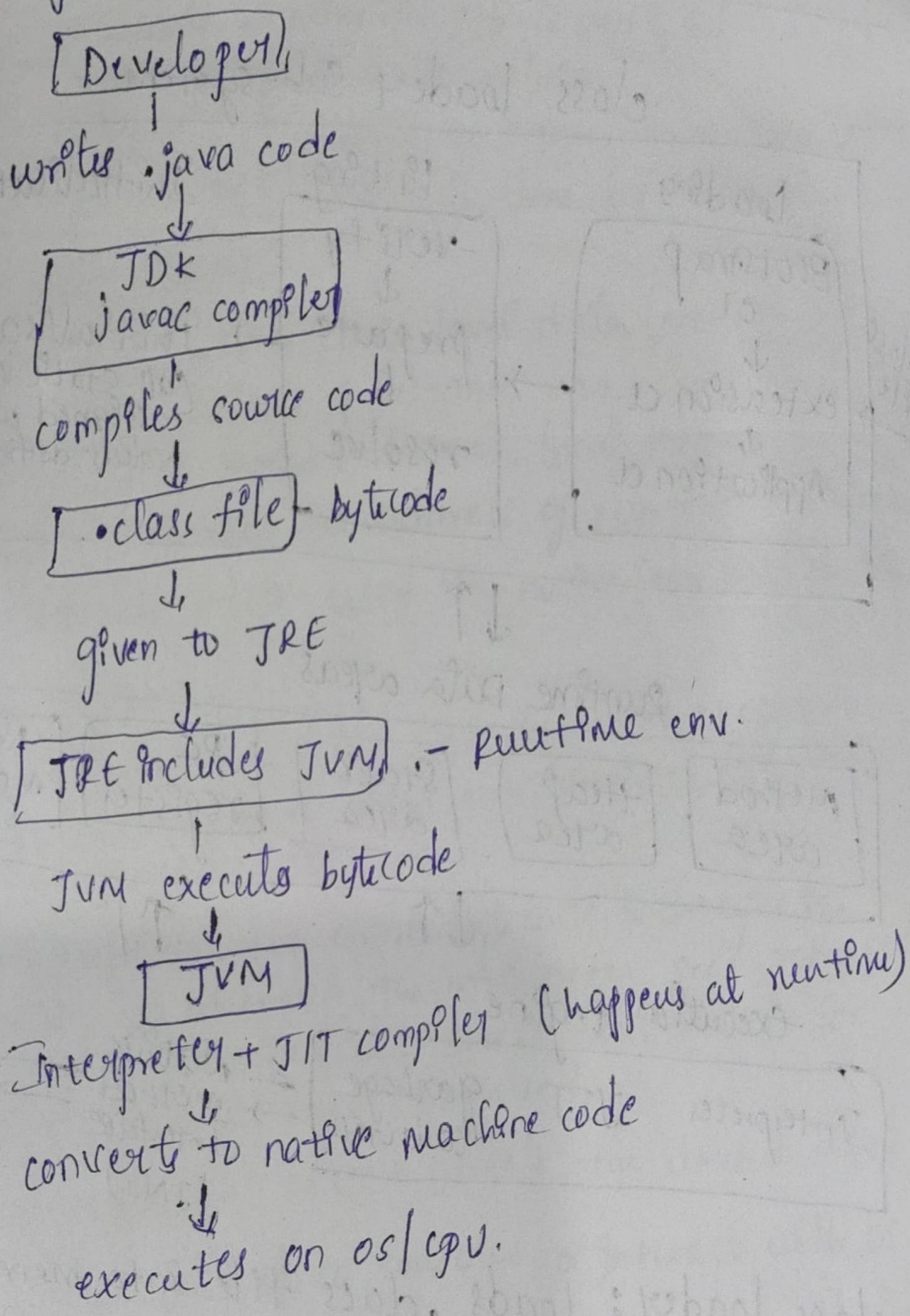
→ JRE + compiler + Debugger + Tools.

→ platform dependent (OS-specific)

→ provide tools to write, compile & run java.



Workflow of JDK, JRE & JVM



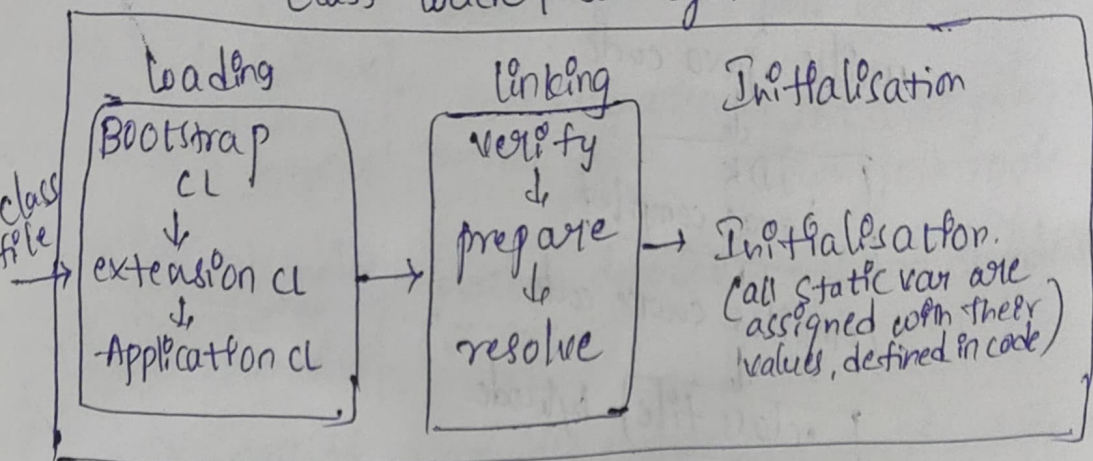
JIT : part of execution engine of JVM.

role

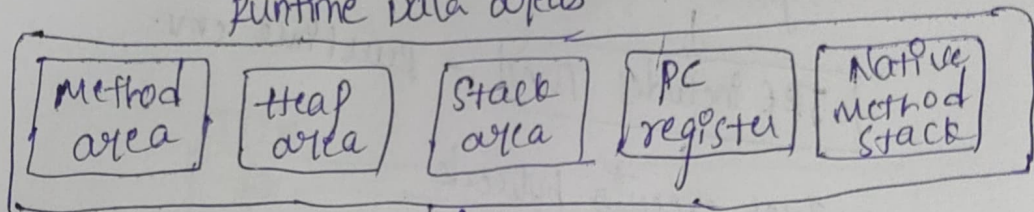
- Improves performance
- reduces redundant compilation - compiled native code is cached, so it can be reused without re-compilation.
- works at runtime.

Architecture of JVM

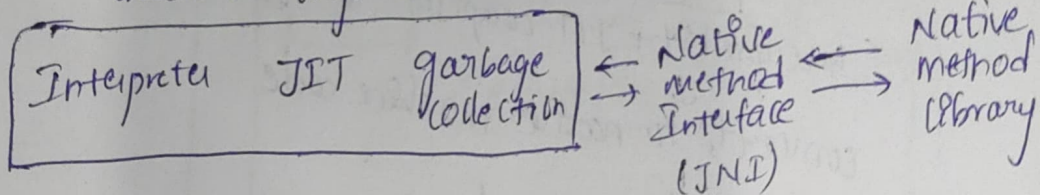
class loader sub system



Runtime Data areas



execution engine



- class loader: loads .class file into memory (method area)
- Bootstrap CL - superclass of extension CL, loads "rt.jar" present in JRE/lib (java.lang, java.util packages).
 - extension CL - loads extension files.
 - Application CL - loads class files from default classpath.

If the class is not found in any class loaders, then we get "runtime exception".

- verify — correctness of file. if fails, throws `VerifyError`
- prepare — allocate memory for static var & initialising the memory to default values.
- resolve — replacing symbolic references from type with direct ref which is done by method area.

Runtime Data Areas

- 1) Method area — stores class level data (methods, static var)
- 2) heap — stores objects & instance var.
- 3) stack — stores method frames, local var.
- 4) pc register — tracks the address of current instruction
- 5) Native method stack — used for native (non-Java) methods.

Execution engine

- 1) Interpretation — line by line bytecode execution.
Dis: when one method is called multiple times, everytime interpretation res.
- 2) JIT — compiles the entire bytecode & changes to native code so whenever the interpreter sees repeated method calls, JIT provides direct native code for that part, so efficiency improved.
- 3) Garbage collector: remove/delete the unused objects.
- 4) JNI — to send op to consoles or interact with OS lib.
- 5) Native libraries — pre-compiled platform-specific code.

Java program structure

```
public class Demo {  
    public static void main (String args[]) {  
        System.out.println("Hello world");  
    }  
}
```


public
↓
Access modifier

static
keyword
↓
w/o creating obj we can access by class name.

void
return type

main (string args[])
↓
method
↓
start of execution

Array of string type.

recognised by

main() : entry point of execution recognised by JVM.

keywords : reserved words (predefined)

→ cannot use them as var, class, method, etc.

Ex. 9: `int age = 25; // int keyword.`

```
int age = 25; // int keyword
public class Hello { // public, class keywords
    // ...
}
```

keywords $\rightarrow$ 59 $\leftarrow$ 48 used $\Rightarrow$ literals (true, false, null)
2 unused (goto, const).

→ class, abstract, break, continue, final, super, static, this, try, throws, finally, void, new etc.

Identifiers - used for identification, it can be variable, class name, method name. etc.

Rules

- Rules
- must begin with (A-Z) (a-z), \$, _ (underscore).
- should not start with digits (0-9).
- case sensitive (name $\neq$ NAME).
- cannot be a keyword (java).

```
eg: public class Demo {  
    psum (String args[]) {  
        int a = 20;
```

Identifiers: classname (demo), method name (main),
string, args, var name (a).

primitive data types → 8, store values directly in memory.

<u>Data type</u>	<u>Default value</u>	<u>Size</u>	<u>range</u>
boolean	false	1 bit	true, false.
byte	0	8 bits (1 byte)	-128 to 127
short	0	2 byte	-32,768 to 32,767
char	\u0000 (unicode)	2 byte	0 to 65535
int	0	4 byte	-2147483648 to 2147483647
long	0	8 byte	-2^{63} to $2^{63}-1$
float	0.0	4 byte	-3.4e38 to 3.4e38
double	0.0	8 byte	-1.7×10^{38} to $1.7 \times 10^{38}-1$

Referenced Dtypes (default value - null)

→ will contain a memory address of var value, becoz the ref types won't store value directly into memory.

• String, Arrays, classes, Interfaces, Objects.

declare var of all primitive Dtypes

byte b = 100;

short s = 2000;

int i = 5010;

long l = 1234567890L;

float f = 5.75f;

double d = 17.99;

char c = 'A';

boolean flag = true;

// program to ip & display stud name, age & grade

```
import java.util.Scanner;
public class student {
    public static void main (String args[]) {
        Scanner sc = new Scanner (System.in);
        String sname = sc.nextLine();
        int age = sc.nextInt();
        char grade = sc.next().charAt(0);
        System.out.println ("Name: " + sname);
        System.out.println ("Age: " + age);
        System.out.println ("Grade: " + grade);
    }
}
```

Implicit casting

- widening casting
- performed by JVM
- No risk of data loss
- int → long, double → float
- No cast needed.

eg: int a = 10;

double b = a;

// Implicit (int to double).

byte → short → char → int → long,
double ← float ←

Explicit casting

- Narrowing casting
- Done manually by programmer.
- risk of data loss.
- ⇒ double → int, long → int
- cast needed: (int) value

eg: double x = 12.56;

int y = (int) x;

// explicit (double to int)

→ y = 12 (decimal loss).

byte ← short ← char ← int
double → float → long

What happens when narrowing conversion fails?

→ primitive type narrowing conversion.

- will not throw exceptions but may result in "data loss, wrap-around, or precision loss" if the value is out of range for target type.

eg: `int i = 300;`

`byte b = (byte) i; b = 44 (300 % 256) // data loss.`

→ Reference narrowing (down casting).

- will throw "ClassCastException" at runtime if the object being cast is not compatible with target type.

eg: `Object obj = new String("hello");`

`Integer num = (Integer) obj; // throws exception.`

Assignment operator (=)

→ Assigns value to var.

eg: `int a = 10;`

Equality operator (==)

→ compares two values

eg: `if (a == 10) {`

// program using arithmetic & relational operators.

```
public class Demo {
```

```
    psum (String args[]) {
```

```
        int a = 10, b = 3;
```

```
        sop (a + b); // 13
```

```
        sop (a - b); // 7
```

```
        sop (a * b); // 30
```

```
        sop (a / b); // 3
```

```
        sop (a % b); // 1
```

```
    // relational
```

```
        sop (a == b); // true
```

```
        sop (a > b); // true
```

```
        sop (a <= b); // false.
```


logical operators && (AND), || (OR), !(NOT)

eg: int a=7, b=4;

soP (a > 5 && b < 10); // true

soP (a < 5 || b > 2); // true

soP (a == b); // true.

Control flow

program to find largest of 3 numbers.

```
public class Demo {
```

```
    ps vm (String args[]) {
```

```
        int a = 10, b = 3, c = -5;
```

```
        int large;
```

```
        if (a >= b && a >= c) {
```

```
            large = a;
```

```
        else if (b >= a && b >= c) {
```

```
            large = b;
```

```
        else {
```

```
            large = c;
```

```
        }
        soP (large);
```

```
    }
```

// print numbers from 10 to 1

```
for (int i = 10; i >= 1; i--) {
```

```
    soP (i);
```

```
}
```

```
int i = 10;
```

```
while (i >= 1) {
```

```
    soP (i);
```

```
    i--;
```

```
}
```


Switch (selection statement)

Instead of writing many if-else statements, you can use the switch statement. It "selects" one of many code blocks to be executed.

eg: int choice = 3;

```
switch (choice) {
```

```
    case 1:
```

```
        sop("choose one");
```

```
        break;
```

```
    case 2:
```

```
        sop("choose two");
```

```
        break;
```

```
    case 3:
```

```
        sop("you choose three");
```

```
        break;
```

```
    default:
```

```
        sop("Invalid choice");
```

```
}
```

Arrays: Collection of elements of same data type stored in contiguous memory location. Allows indexed access to elements.

types: 1D $\rightarrow$ int[] arr = new int[5];

2D $\rightarrow$ int[][] arr = new int[2][3];

multidimensional $\rightarrow$ Array of arrays of arrays...

// sum of elements in an array.

```
public class calculate {
```

```
    psum (String args[]) {
```

```
        int int[] arr = {10, 20, 30};
```

```
        int sum = 0;
```

```
        for (int i = 0; i < arr.length; i++) {
```

```
            sum += arr[i];
```

```
        }
```

```
        sop(sum); // 60
```

```
}
```

// Declare & Initialise a 2D array & print elements.

```
public class twoDarray {
```

```
    psvm (String args[]) {
```

```
        int[][] arr = { {1,2,3}, {4,5,6} };
```

```
        sop ("2D array elements: ");
```

```
        for (int i=0; i<arr.length; i++) { // rows.
```

```
            for (int j=0; j<arr[i].length; j++) { // columns
```

```
                sop (arr[i][j] + " ");
```

```
            }  
            sopln (); // new line after each row
```

// Input from user. & print elements of array.

```
import java.util.*;
```

```
public class Demo {
```

```
    psvm (String args[]) {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        int n = sc.nextInt();
```

```
        int[] arr = new int [n];
```

```
        for (int i=0; i<arr.length; i++) {
```

```
            arr[i] = sc.nextInt();
```

```
        }
```

```
        for (int i=0; i<n; i++) {
```

```
            sop (arr[i]);
```

```
        }
```

```
    }
```